

This lab session covers Django Forms (Part 2 – Django form field and file handling). By the end of this session, students should have a very good understanding of how to use Django form fields.

Pre-lab Preparation:

1. The table student and address as in Lab (8).
2. The fake data from mockaroo.com website or any other methods.

Lab Activities:

Task 1: Create a URL to list, add, update, or delete students and specify addresses (one address for each student) by applying the Django form field with any necessary HTML files and view functions.

```
class Address(models.Model):
    city = models.CharField(max_length=100)

    def __str__(self):
        return self.city

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    address = models.ForeignKey(Address, on_delete=models.CASCADE)

    def __str__(self):
        return self.name
```

```
class StudentForm(forms.ModelForm):

    class Meta:
        model = Student
        fields = '__all__'
```

```
def lab11_edit_student(request, id):

    student = Student.objects.get(id=id)

    if request.method == 'POST':

        form = StudentForm(request.POST, instance=student)

        if form.is_valid():

            form.save()

            return redirect('/books/lab11/students/')

    else:

        form = StudentForm(instance=student)

    return render(
        request,
        'bookmodule/lab11_add_student.html',
        {'form': form}
    )

def lab11_delete_student(request, id):

    student = Student.objects.get(id=id)

    student.delete()

    return redirect('/books/lab11/students/')
```

```
def lab11_students(request):

    students = Student.objects.all()

    return render(
        request,
        'bookmodule/lab11_students.html',
        {'students': students}
    )

def lab11_add_student(request):

    if request.method == 'POST':

        form = StudentForm(request.POST)

        if form.is_valid():

            form.save()

            return redirect('/books/lab11/students/')

    else:

        form = StudentForm()

    return render(
        request,
        'bookmodule/lab11_add_student.html',
        {'form': form}
    )
```

```
path('lab11/students/', views.lab11_students, name='lab11.students'),
path('lab11/addstudent/', views.lab11_add_student, name='lab11.addstudent'),
path('lab11/editstudent/<int:id>', views.lab11_edit_student, name='lab11.editstudent'),
path('lab11/deletestudent/<int:id>', views.lab11_delete_student, name='lab11.deletestudent'),
]
```

```
apps > templates > bookmodule > <> lab11_add_student.html > ...
1  {% extends "layouts/base.html" %}
2
3  {% block content %}
4
5  <h1>Add Student</h1>
6
7  <form method="POST">
8      {% csrf_token %}
9      {{ form.as_p }}
10
11      <button type="submit">
12          Save
13      </button>
14
15  </form>
16
17  {% endblock %}
```

```
apps > templates > bookmodule > <> lab11_students.html > a
1  {% extends "layouts/base.html" %}
2
3  {% block content %}
4
5  <h1>Students List</h1>
6
7  <a href="{% url 'lab11.addstudent' %}">Add Student</a>
8  <hr>
9
10  {% for student in students %}
11
12      <h3>{{ student.name }}</h3>
13
14      <p>Age: {{ student.age }}</p>
15
16      <p>City: {{ student.address.city }}</p>
17
18      <a href="{% url 'lab11.editstudent' student.id %}">
19          Edit
20      </a>
21
22      <a href="{% url 'lab11.deletestudent' student.id %}">
23          Delete
24      </a>
25
26  <hr>
27
28  {% endfor %}
29
30  {% endblock %}
```

[Home](#) [List](#) [Books](#) [About](#)

Students List

[Add Student](#)

Deema

Age: 23

City: Riyadh

[Edit](#) [Delete](#)

Leen

Age: 21

City: Arrass

[Edit](#) [Delete](#)

Lamo

Age: 26

City: Riyadh

[Edit](#) [Delete](#)

ghado

Age: 26

City: Arrass

[Edit](#) [Delete](#)

Reema

Task 2: Suppose the relation between students and addresses is many-to-many, in which a student can have multiple addresses, and also, one address can be associated with many students. In this task, the code in the previous task should be copied and modified to implement many-to-many relationships.

Note: If needed, student or address models should be copied and renamed to, for example, student2 and address2.

```
(djangoenv) fridy@MacBook-Air-Fridy CS471-Lab4 % python3 manage.py makemigrations
Migrations for 'bookmodule':
  apps/bookmodule/migrations/0004_address2_student2.py
  + Create model Address2
  + Create model Student2
(djangoenv) fridy@MacBook-Air-Fridy CS471-Lab4 % python3 manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, bookmodule, contenttypes, sessions
Running migrations:
  Applying bookmodule.0004_address2_student2... OK
(djangoenv) fridy@MacBook-Air-Fridy CS471-Lab4 %
```

```
class Student2Form(forms.ModelForm):

    class Meta:
        model = Student2
        fields = '__all__'
```

```
class Address2(models.Model):
    city = models.CharField(max_length=100)

    def __str__(self):
        return self.city

class Student2(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    addresses = models.ManyToManyField(Address2)

    def __str__(self):
        return self.name
```

```
1 {% extends "layouts/base.html" %}
2
3 {% block content %}
4
5 <h1>Add Student</h1>
6
7 <form method="POST">
8
9     {% csrf_token %}
10
11     {{ form.as_p }}
12
13     <button type="submit">
14         Save
15     </button>
16
17 </form>
18
19 {% endblock %}
```

```
apps > templates > bookmodule > > lab11_task2_students.html > ...
1 {% extends "layouts/base.html" %}
2
3 {% block content %}
4
5 <h1>Task 2 Students</h1>
6
7 <a href="/books/lab11/task2/addstudent/">
8     Add Student
9 </a>
10
11 <hr>
12
13 {% for student in students %}
14
15     <h3>{{ student.name }}</h3>
16
17     <p>Age: {{ student.age }}</p>
18
19     <p>Addresses:</p>
20
21     <ul>
22         {% for address in student.addresses.all %}
23             <li>{{ address.city }}</li>
24         {% endfor %}
25     </ul>
26
27     <a href="/books/lab11/task2/editstudent/{{ student.id }}">
28         Edit
29     </a>
30
31     <a href="/books/lab11/task2/deletestudent/{{ student.id }}">
32         Delete
33     </a>
34
35     <hr>
36
37 {% endfor %}
38
39 {% endblock %}
```

Task 3: Create an additional table (of your own choice) that includes images, and you should build forms to handle data with images through Django.

Note: A few configurations are needed for file handling.

```
class ProductImage(models.Model):  
    name = models.CharField(max_length=100)  
    image = models.ImageField(upload_to='images/')  
    def __str__(self):  
        return self.name
```

```
(djangoenv) fridy@MacBook-Air-Fridy CS471-Lab4 % pip install pillow  
Collecting pillow  
  Downloading pillow-12.2.0-cp314-cp314-macosx_11_0_arm64.whl.metadata (8.8 kB)  
  Downloading pillow-12.2.0-cp314-cp314-macosx_11_0_arm64.whl (4.7 MB)  
    4.7/4.7 MB 2.9 MB/s 0:00:01  
Installing collected packages: pillow  
Successfully installed pillow-12.2.0  
[notice] A new release of pip is available: 26.0.1 -> 26.1.1  
[notice] To update, run: pip install --upgrade pip  
(djangoenv) fridy@MacBook-Air-Fridy CS471-Lab4 %
```

```
29 MEDIA_URL = '/media/'  
30 MEDIA_ROOT = BASE_DIR / 'media'
```

```
class ProductImageForm(forms.ModelForm):  
    class Meta:  
        model = ProductImage  
        fields = '__all__'
```

```
apps > templates > bookmodule > lab11_add_image.html > ...  
1 {% extends "layouts/base.html" %}  
2  
3 {% block content %}  
4  
5 <h1>Add Image</h1>  
6  
7 <form method="POST" enctype="multipart/form-data">  
8  
9     {% csrf_token %}  
10  
11     {{ form.as_p }}  
12  
13     <button type="submit">  
14         Save  
15     </button>  
16  
17 </form>  
18  
19 {% endblock %}
```

```
def lab11_task3_add_image(request):  
    if request.method == 'POST':  
        form = ProductImageForm(request.POST, request.FILES)  
        if form.is_valid():  
            form.save()  
    else:  
        form = ProductImageForm()  
    return render(request,  
        'bookmodule/lab11_add_image.html',  
        {'form': form})
```

```
17 from django.contrib import admin  
18 from django.urls import include, path  
19 from django.conf import settings  
20 from django.conf.urls.static import static  
21  
22 urlpatterns = [  
23     path('admin/', admin.site.urls),  
24     path('books/', include('apps.bookmodule.urls')),  
25     path('users/', include('apps.usermodule.urls')),  
26 ]  
27 urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
(djangoenv) fridy@MacBook-Air-Fridy CS471-Lab4 % python3 manage.py migrate  
Operations to perform:  
  Apply all migrations: admin, auth, bookmodule, contenttypes, sessions  
Running migrations:  
  Applying bookmodule.0005_productimage... OK  
(djangoenv) fridy@MacBook-Air-Fridy CS471-Lab4 % python3 manage.py runserver  
Watching for file changes with StatReloader  
Performing system checks...  
  
System check identified no issues (0 silenced).  
May 17, 2026 - 20:47:40  
Django version 6.0.4, using settings 'libraryproject.settings'  
Starting development server at http://127.0.0.1:8000/  
Quit the server with CONTROL-C.  
  
WARNING: This is a development server. Do not use it in a production setting. Use a production server instead.  
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
```